Tree-based methods

Data Analytics

Jyotishka Datta

2024-02-20

Contents

Decision Trees

Compare to logistic

Tree-based Regression

Bagging and Random Forest

Tree Based Methods

We will show how to grow a tree for classification. Regression trees are developed in a very similar way.

As before, these codes are taken from the Introduction to Statistical Learning book and the Hands-On Machine Learning Book https://bradleyboehmke.github.io/HOML/DT.html (https://bradleyboehmke.github.io/HOML/DT.html).

Load libraries

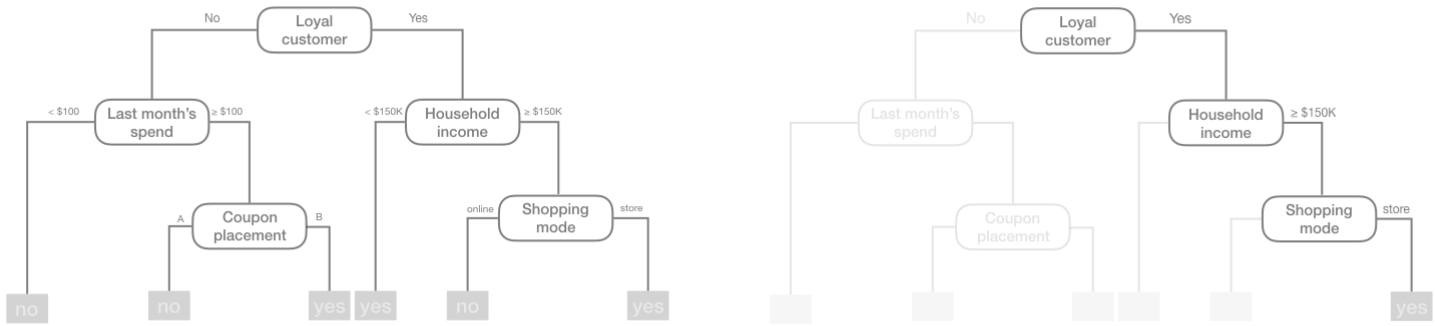
```r
# Helper packages
library(dplyr)       # for data wrangling
library(ggplot2)     # for awesome plotting

# Modeling packages
library(rpart)       # direct engine for decision tree application
library(caret)       # meta engine for decision tree application

# Model interpretability packages
library(rpart.plot)  # for plotting decision trees
library(vip)         # for feature importance
library(pdp)         # for feature effects
```

Decision trees

- Decision trees are a popular machine learning algorithm for both regression and classification problems.
- Decision trees partition the training data into homogeneous subgroups with similar response values and fit a simple constant in each subgroup.
- The subgroups or nodes are formed recursively using binary partitions formed by asking simple yes-or-no questions about each feature.
- This process continues until a stopping criteria is met, such as a maximum tree depth being reached.
- For regression problems, the model predicts the output based on the average response values for all observations that fall in that subgroup.
- For classification problems, the model predicts the class that has majority representation within the subgroup, and predicted probabilities can be obtained using the proportion of each class within the subgroup.

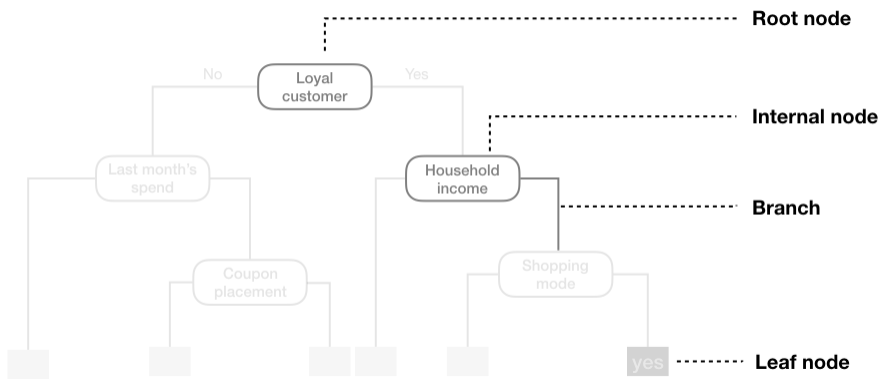
Tree structure

- In essence, our tree is a set of rules that allows us to make predictions by asking simple yes-or-no questions about each feature.
- For example, if the customer is loyal, has household income greater than $150,000, and is shopping in a store, the exemplar tree diagram in Figure 9.1 would predict that the customer will redeem a coupon.



Tree terminology

- A decision tree starts with a single subgroup called the root node, which contains all the training data.
- The final subgroups at the bottom of the tree are called the terminal nodes or leaves.
- Every subgroup between the root and the terminal nodes is referred to as an internal node.
- The branches of the tree represent the connections between nodes.



Recursive partitioning

- CART is a decision tree algorithm that uses binary recursive partitioning, where each split or rule depends on the splits above it.
- At each node, CART aims to find the "best" feature to partition the remaining data into two regions, R_1 and R_2.
- The goal is to minimize the overall error between the actual response (y_i) and the predicted constant (c_i).

$$SSE = \sum_{i \in R_1} (y_i - c_1)^2 + \sum_{i \in R_2} (y_i - c_2)^2$$

- For classification problems, the partitioning is typically made to maximize the reduction in cross-entropy or the Gini index.

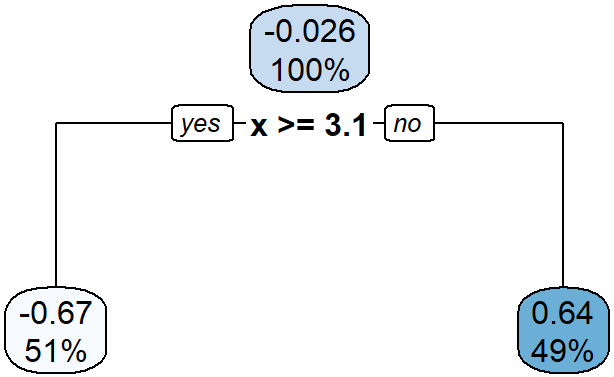
Fitting

- A single predictor or feature (X) can be used multiple times in a tree.
- Let us try with this simulated dataset: $Y_i \overset{iid}{\sim} N\left(\sin(X_i), \sigma^2\right), i = 1, \ldots, n$.

Generating sinusoidal curve and fitting a tree

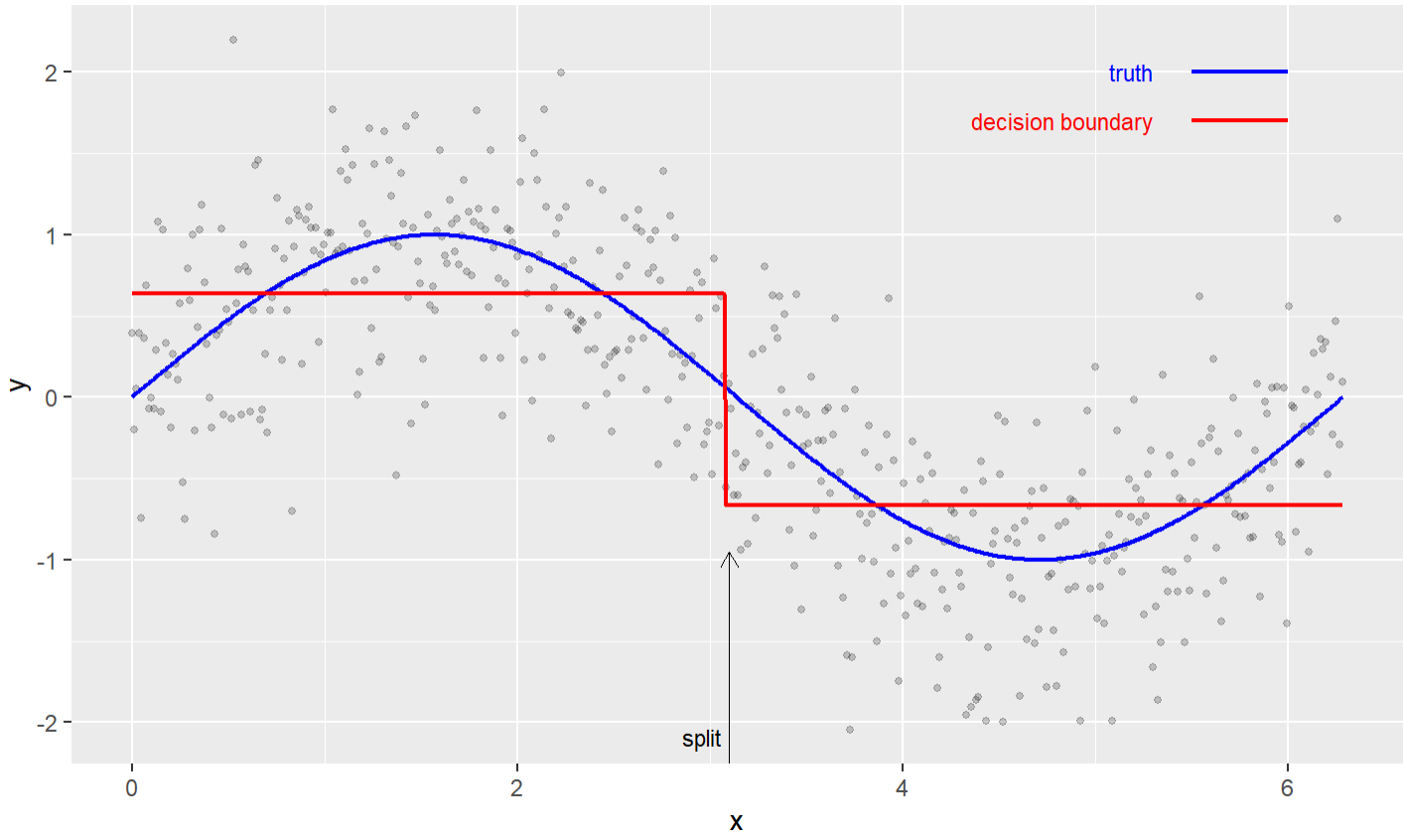
```
# create data
set.seed(1112)  # for reproducibility
df <- tibble::tibble(
  x = seq(from = 0, to = 2 * pi, length = 500),
  y = sin(x) + rnorm(length(x), sd = 0.5),
  truth = sin(x)
)

# run decision stump model
ctrl <- list(cp = 0, minbucket = 5, maxdepth = 1)
fit <- rpart(y ~ x, data = df, control = ctrl)

# plot tree
par(mar = c(1, 1, 1, 1))
rpart.plot(fit)
```

Generating sinusoidal curve and fitting a tree



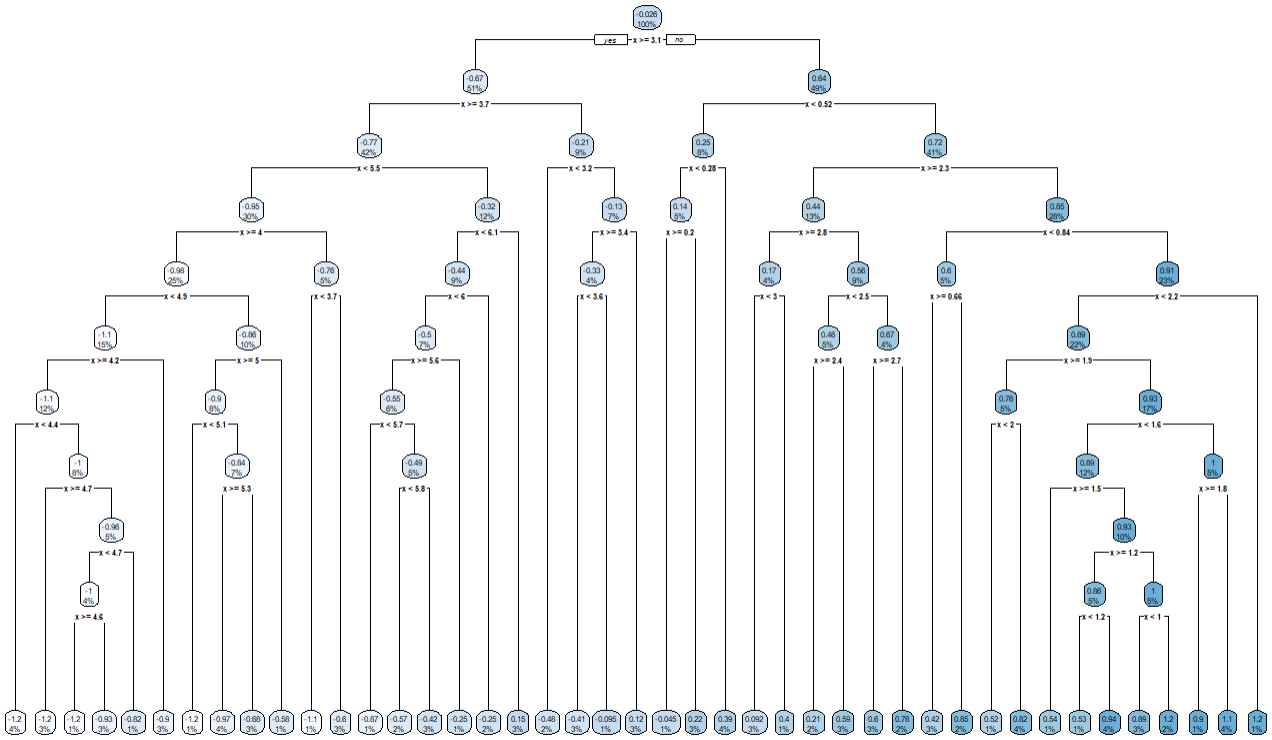
Plot decision boundary

- The resulting decision boundary illustrates the predicted value when x < 3.1 (0.64), and when x > 3.1.



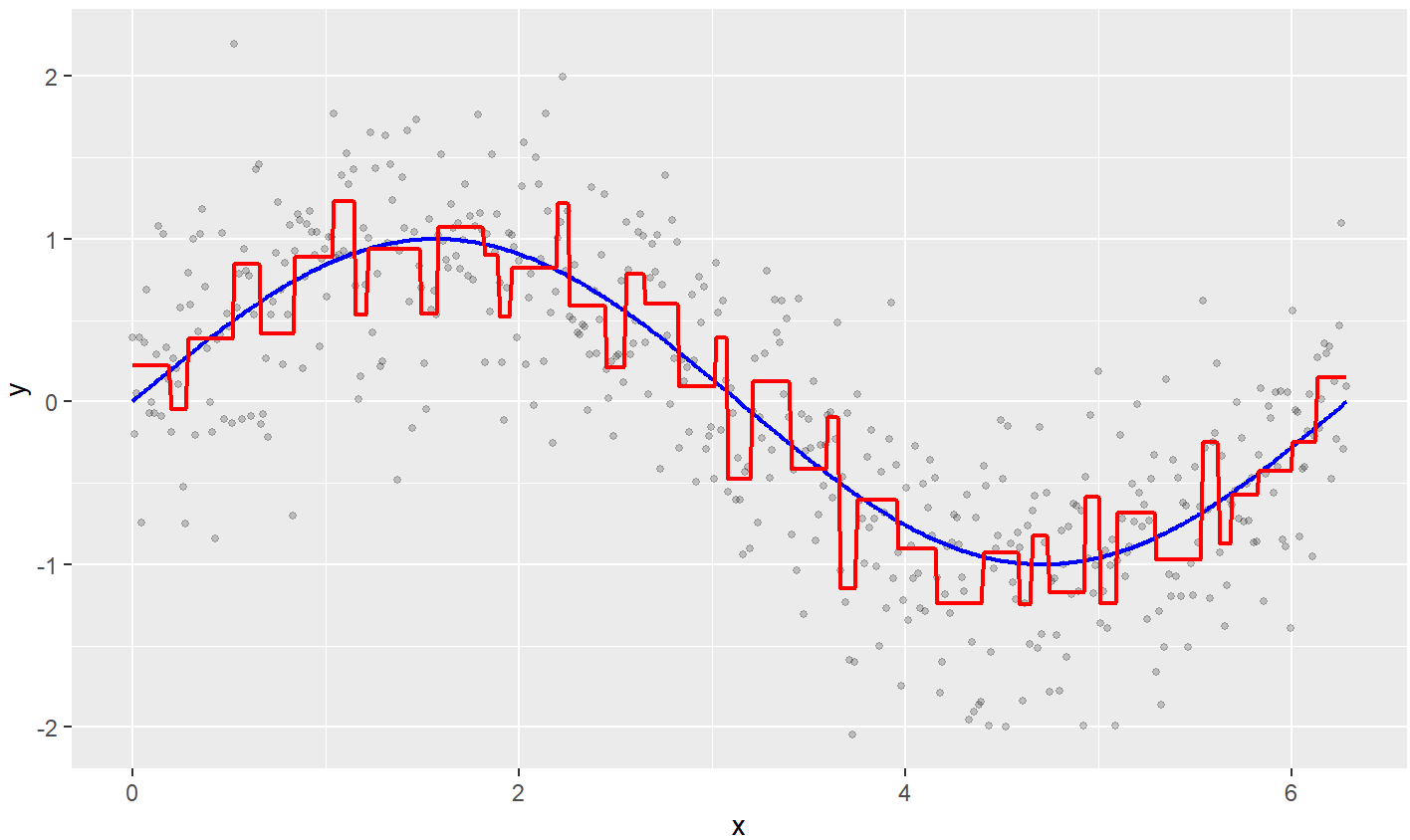
Now, you can go a little deeper

```
# fit depth 10 decision tree
ctrl <- list(cp = 0, minbucket = 5, maxdepth = 10)
fit <- rpart(y ~ x, data = df, control = ctrl)
rpart.plot(fit)
```



Plot the boundary, too

As we build a deeper tree, the algorithm will repeatedly split the data based on the same feature since it is the only one available for partitioning. The algorithm will continue to find the optimal splits along the values of this feature until a pre-determined stopping criteria is met.



The `tree` library

- There is also a `tree` library that can fit a decision tree to your data.
- In my opinion, it's not as polished but it's worth taking a look.
- We will analyze a `Carseats` data with this first.

Carseats data

In these data, Sales is a continuous variable, and so we begin by recoding it as a binary variable.

```
set.seed(2441139)
library(tree)
library(ISLR)
attach(Carseats)
High=ifelse(Sales<=8,"No","Yes")
High=as.factor(ifelse(Sales<=8,"No","Yes"))

Carseats$High <- High
```

Fit a tree

```
tree.carseats=tree(High~.-Sales,Carseats)
summary(tree.carseats)
```

```
##
## Classification tree:
## tree(formula = High ~ . - Sales, data = Carseats)
## Variables actually used in tree construction:
## [1] "ShelveLoc"   "Price"       "Income"      "CompPrice"   "Population"
## [6] "Advertising" "Age"         "US"
## Number of terminal nodes:  27
## Residual mean deviance:  0.4575 = 170.7 / 373
## Misclassification error rate: 0.09 = 36 / 400
```

Interpretaion

We see that the training error rate is 9%.

For classification trees, the deviance reported in the output of summary() is given by

$$-2 \sum_m \sum_k n_{mk} \log(\hat{p}_{mk})$$

where n_{mk} is the number of observations in the mth terminal node that belong to the kth class.

A small deviance indicates a tree that provides a good fit to the (training) data.

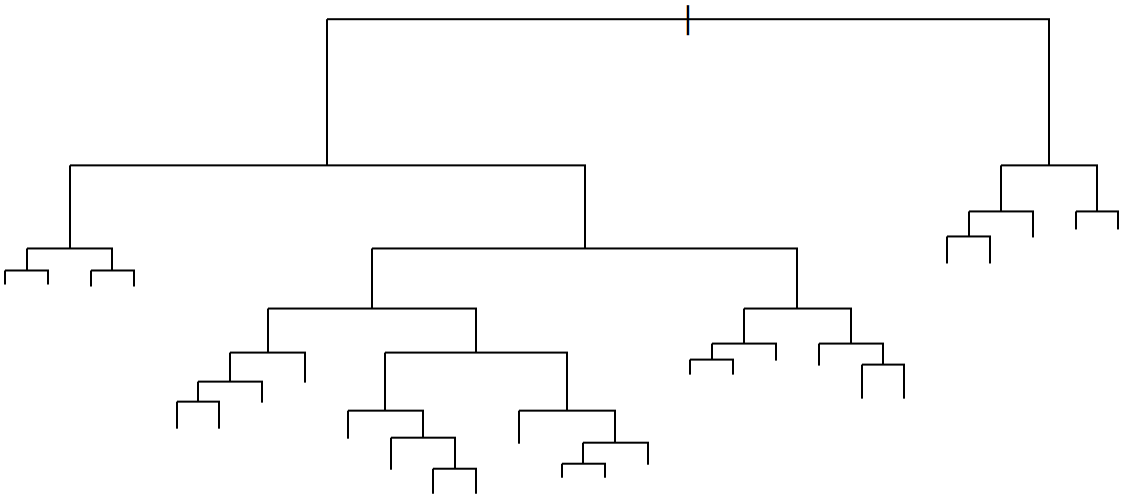
The residual mean deviance reported is simply the deviance divided by $n - |T_0|$, which in this case is 400 - 27 = 373.

Plotting the tree

- One of the most attractive properties of trees is that they can be graphically displayed.
- We use the `plot()` function to display the tree structure, and the `text()` function to display the node labels.

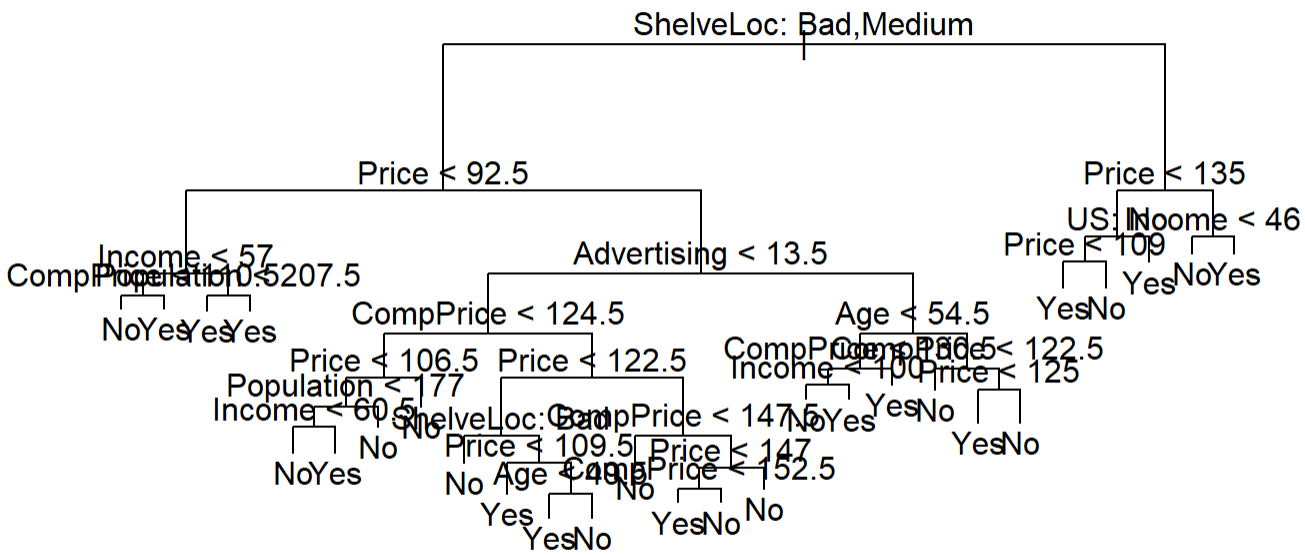
Plot tree

```
plot(tree.carseats)
```



Plot tree

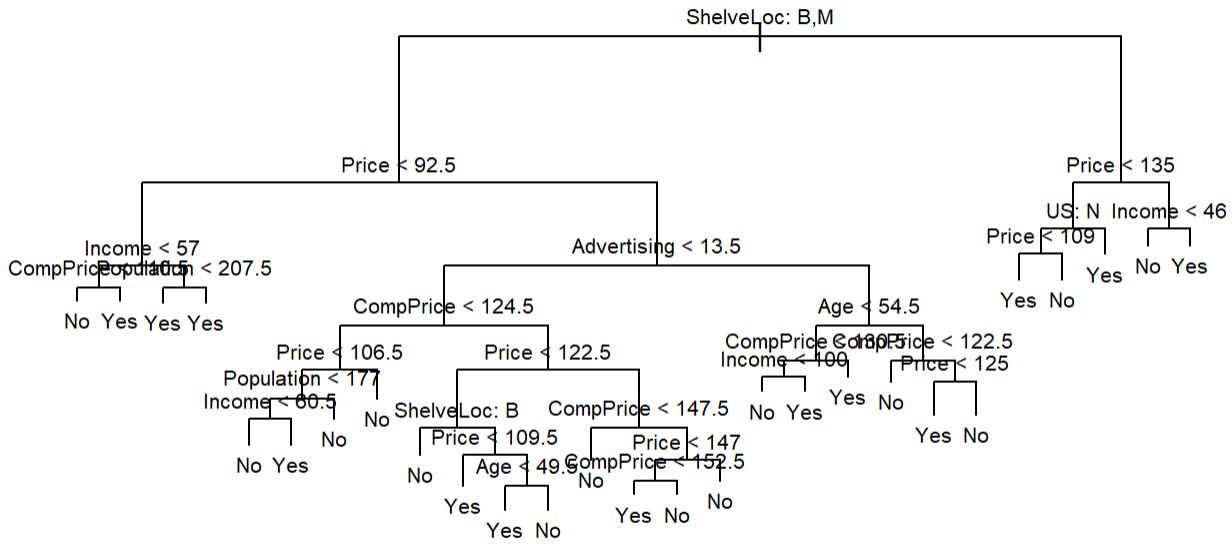
```
plot(tree.carseats)
text(tree.carseats,pretty=0)
```



Plot tree

Shelving location is the most important predictor here.

```
plot(tree.carseats)
text(tree.carseats,pretty=1, cex = 0.65)
```



Tree output

- If we just type the name of the tree object, `R` prints output corresponding to each branch of the tree.
- `R` displays the split criterion (e.g. `Price<92.5`), the number of observations in that branch, the deviance, the overall prediction for the branch (`Yes` or `No`), and the fraction of observations in that branch that take on values of `Yes` and `No`.

Tree output in R

```
tree.carseats
```

```
## node), split, n, deviance, yval, (yprob)
##       * denotes terminal node
##
##   1) root 400 541.500 No ( 0.59000 0.41000 )
##     2) ShelveLoc: Bad,Medium 315 390.600 No ( 0.68889 0.31111 )
##       4) Price < 92.5 46  56.530 Yes ( 0.30435 0.69565 )
##         8) Income < 57 10  12.220 No ( 0.70000 0.30000 )
##          16) CompPrice < 110.5 5   0.000 No ( 1.00000 0.00000 ) *
##          17) CompPrice > 110.5 5   6.730 Yes ( 0.40000 0.60000 ) *
##         9) Income > 57 36  35.470 Yes ( 0.19444 0.80556 )
##          18) Population < 207.5 16  21.170 Yes ( 0.37500 0.62500 ) *
##          19) Population > 207.5 20   7.941 Yes ( 0.05000 0.95000 ) *
##       5) Price > 92.5 269 299.800 No ( 0.75465 0.24535 )
##        10) Advertising < 13.5 224 213.200 No ( 0.81696 0.18304 )
##          20) CompPrice < 124.5 96  44.890 No ( 0.93750 0.06250 )
##            40) Price < 106.5 38  33.150 No ( 0.84211 0.15789 )
##              80) Population < 177 12  16.300 No ( 0.58333 0.41667 )
##               160) Income < 60.5 6   0.000 No ( 1.00000 0.00000 ) *
##               161) Income > 60.5 6   5.407 Yes ( 0.16667 0.83333 ) *
##              81) Population > 177 26   8.477 No ( 0.96154 0.03846 ) *
##            41) Price > 106.5 58   0.000 No ( 1.00000 0.00000 ) *
##          21) CompPrice > 124.5 128 150.200 No ( 0.72656 0.27344 )
##            42) Price < 122.5 51  70.680 Yes ( 0.49020 0.50980 )
##              84) ShelveLoc: Bad 11   6.702 No ( 0.90909 0.09091 ) *
##              85) ShelveLoc: Medium 40  52.930 Yes ( 0.37500 0.62500 )
##               170) Price < 109.5 16   7.481 Yes ( 0.06250 0.93750 ) *
##               171) Price > 109.5 24  32.600 No ( 0.58333 0.41667 )
##                 342) Age < 49.5 13  16.050 Yes ( 0.30769 0.69231 ) *
##                 343) Age > 49.5 11   6.702 No ( 0.90909 0.09091 ) *
##            43) Price > 122.5 77  55.540 No ( 0.88312 0.11688 )
```

Training and Testing

- Need test error for proper evaluation.
- Split the observations into a training set and a test set, *build the tree using the training set*, and evaluate its performance on the test data.
- The `predict()` function can be used for this purpose.
- In the case of a classification tree, the argument `type="class"` instructs `R` to return the actual class prediction.

Test error

```
set.seed(2)
train=sample(1:nrow(Carseats), 200)
Carseats.test=Carseats[-train,]
High.test=High[-train]
tree.carseats=tree(High~.-Sales,Carseats,subset=train)
tree.pred=predict(tree.carseats,Carseats.test,type="class")
table(tree.pred,High.test)
```

```
##          High.test
## tree.pred  No Yes
##       No  104  33
##       Yes  13  50
```

This approach leads to correct predictions for 0.715 cases.

Pruning

- What you just saw was the full tree.
- Next, we consider whether pruning the tree might lead to improved results.
- The function `cv.tree()` performs cross-validation in order to determine the optimal level of tree complexity; **cost complexity pruning** is used in order to select a sequence of trees for consideration.

Recap (Classification Losses)

1. The 0-1 loss or misclassification error rate:

$$\sum_{m=1}^{|T|} \sum_{x_i \in R_m} 1(y_i \neq y_{R_m})$$

2. Cross-entropy (measures the ‘purity’ of a leaf)

$$-\sum_{m=1}^{|T|} q_m \sum_{k=1}^{K} \hat{p}_{mk} \log(\hat{p}_{mk})$$

where $\hat{p}_{mk}$ is the proportion of class k within R_m, and q_m is the proportion of samples in R_m.

- Use cross-entropy for growing the tree, while using the misclassification rate when pruning the tree.

Pruning (continued)

- We use the argument `FUN=prune.misclass` in order to indicate that we want the *classification error rate* to guide the cross-validation and pruning process, rather than the default for the `cv.tree()` function, which is deviance.

- The `cv.tree()` function reports

1. the number of terminal nodes of each tree considered (size),
2. the corresponding error rate, and
3. the value of the cost-complexity parameter used (`k` corresponds to our α)

Cost-complexity pruning

- Cost-complexity pruning is similar to regularization idea. You put a penalty for the size of the tree - larger trees are penalized more.

$$\text{minimize} \sum_{m=1}^{|T|} \sum_{x_i \in R_m} 1(y_i \neq y_{R_m}) + \alpha |T|$$

- Just like regularized methods, α is a tuning parameter - we can choose a suitable value by cross-validation.

Using `cv.tree()`

```
set.seed(3)
cv.carseats=cv.tree(tree.carseats,FUN=prune.misclass)
names(cv.carseats)
```

```
## [1] "size"   "dev"    "k"      "method"
```

```
cv.carseats
```

```
## $size
## [1] 21 19 14  9  8  5  3  2  1
##
## $dev
## [1] 74 76 81 81 75 77 78 85 81
##
## $k
## [1] -Inf  0.0  1.0  1.4  2.0  3.0  4.0  9.0 18.0
##
## $method
## [1] "misclass"
##
## attr(,"class")
## [1] "prune"         "tree.sequence"
```

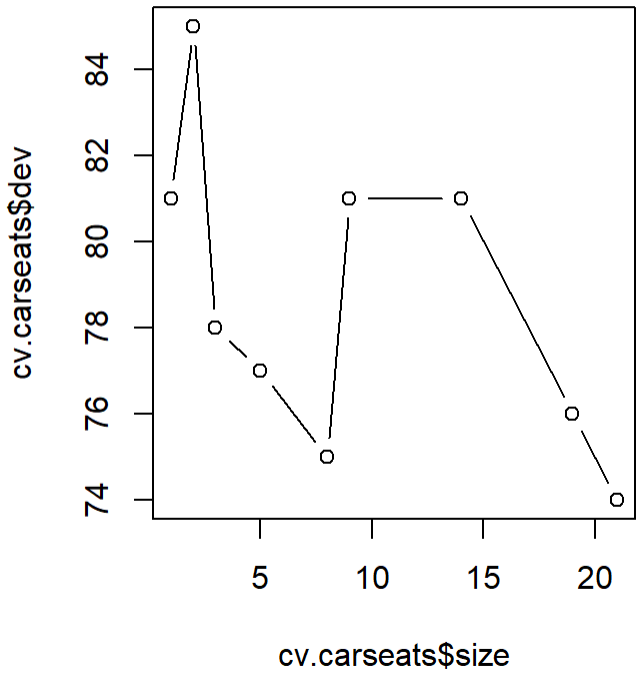
Output explanation

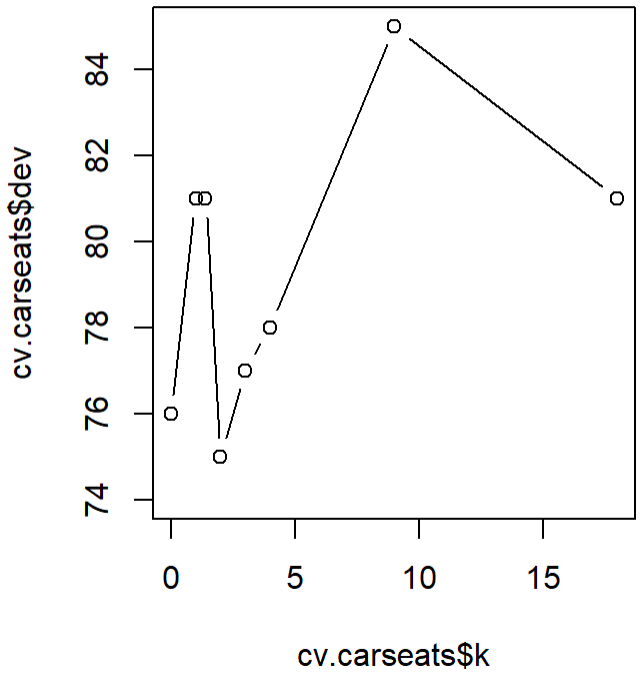
- Note that, despite the name, `dev` corresponds to the cross-validation error rate in this instance.
- The tree with 9 terminal nodes results in the lowest cross-validation error rate, with 50 cross-validation errors.

The tree with 9 terminal nodes has the same misclassification rate as the one with 21 nodes.

Plotting CV errors

```
par(mfrow=c(1,2))
plot(cv.carseats$size,cv.carseats$dev,type="b")
plot(cv.carseats$k,cv.carseats$dev,type="b")
```

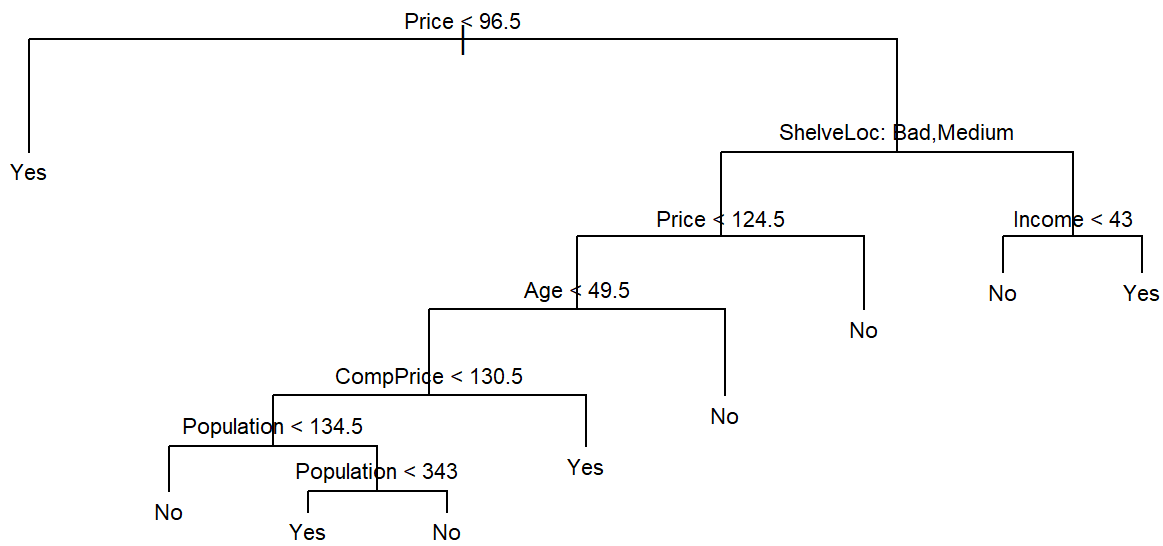




Prune the tree

We now apply the `prune.misclass()` function in order to prune the tree to obtain the nine-node tree.

```
prune.carseats=prune.misclass(tree.carseats,best=9)
plot(prune.carseats)
text(prune.carseats,pretty=0, cex = 0.7)
```



Pruned tree

- How does the pruned tree do in terms of prediction?

```
tree.pred=predict(prune.carseats,Carseats.test,type="class")
table(tree.pred,High.test)
```

```
##          High.test
## tree.pred No Yes
##       No  97  25
##       Yes 20  58
```

Now precentage of correct prediction is 0.77.

Different k

- You can choose a different k, e.g. $k = 15$.

```
prune.carseats=prune.misclass(tree.carseats,best=15)
tree.pred=predict(prune.carseats,Carseats.test,type="class")
table(tree.pred,High.test)
```

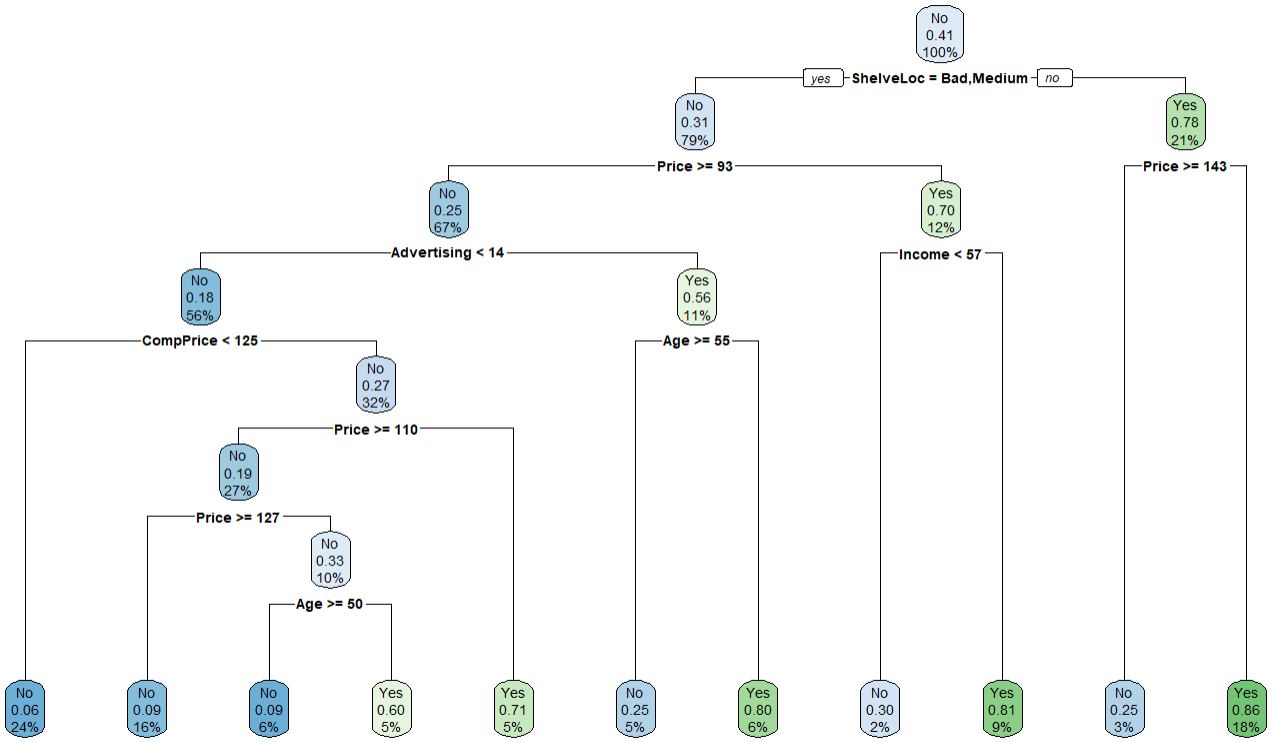
```
##          High.test
## tree.pred  No Yes
##       No  102  30
##       Yes  15  53
```

- Percentage of correct predictions will be lower: 0.74
- Compare to the best model by CV: 0.77

Now the `rpart` function we used before

```
set.seed(12345)
rpart.seats <- rpart(High~.-Sales, data = Carseats)
rpart.plot(rpart.seats)
```

Final tree from rpart() is of size 11.



The fitted object

- Just like the `tree` function, we can take a peak at the decision tree output.
- This prints various information about the different splits.

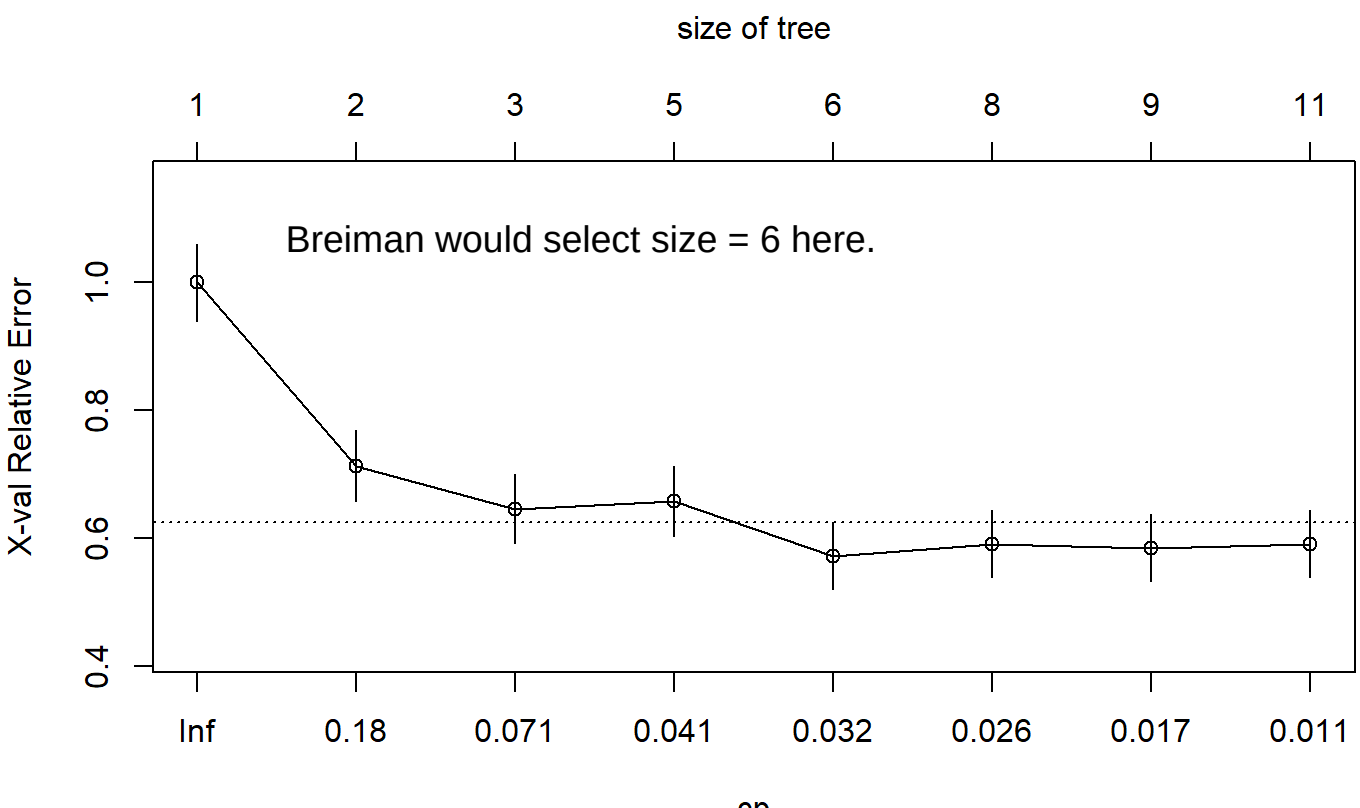
```
rpart.seats
```

```
## n= 400
##
## node), split, n, loss, yval, (yprob)
##       * denotes terminal node
##
##   1) root 400 164 No (0.59000000 0.41000000)
##     2) ShelveLoc=Bad,Medium 315  98 No (0.68888889 0.31111111)
##       4) Price>=92.5 269  66 No (0.75464684 0.24535316)
##         8) Advertising< 13.5 224  41 No (0.81696429 0.18303571)
##          16) CompPrice< 124.5 96   6 No (0.93750000 0.06250000) *
##          17) CompPrice>=124.5 128  35 No (0.72656250 0.27343750)
##            34) Price>=109.5 107  20 No (0.81308411 0.18691589)
##              68) Price>=126.5 65   6 No (0.90769231 0.09230769) *
##              69) Price< 126.5 42  14 No (0.66666667 0.33333333)
##               138) Age>=49.5 22   2 No (0.90909091 0.09090909) *
##               139) Age< 49.5 20   8 Yes (0.40000000 0.60000000) *
##            35) Price< 109.5 21   6 Yes (0.28571429 0.71428571) *
##         9) Advertising>=13.5 45  20 Yes (0.44444444 0.55555556)
##          18) Age>=54.5 20   5 No (0.75000000 0.25000000) *
##          19) Age< 54.5 25   5 Yes (0.20000000 0.80000000) *
##       5) Price< 92.5 46  14 Yes (0.30434783 0.69565217)
##        10) Income< 57 10   3 No (0.70000000 0.30000000) *
##        11) Income>=57 36   7 Yes (0.19444444 0.80555556) *
##     3) ShelveLoc=Good 85  19 Yes (0.22352941 0.77647059)
##       6) Price>=142.5 12   3 No (0.75000000 0.25000000) *
```

Tuning in `rpart`

- Behind the scenes `rpart()` is automatically applying a range of cost complexity values to prune the tree.
- To compare the error for each α value, `rpart()` performs a 10-fold CV (by default).
- Breiman (1984) suggested that in actual practice, it's common to instead use the smallest tree within 1 standard error (SE) of the minimum CV error (this is called the 1-SE rule).

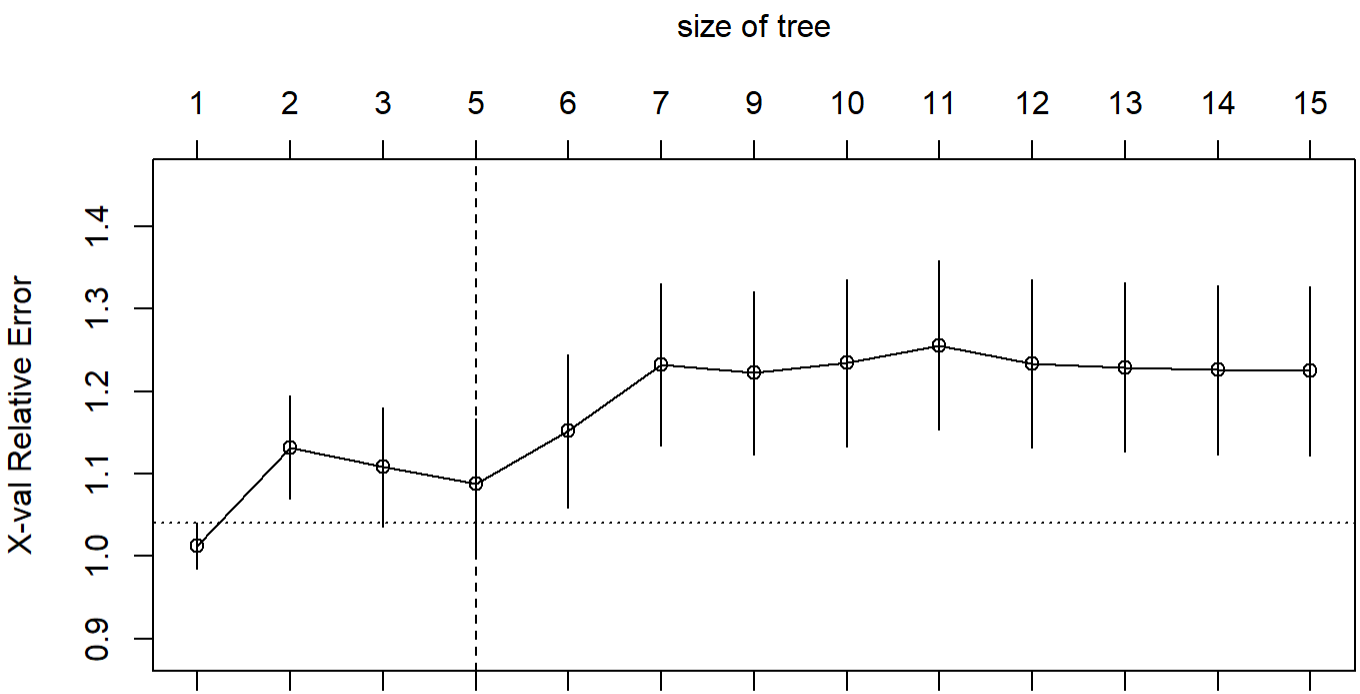
```
set.seed(12345)
plotcp(rpart.seats)
```



Full tree and tuning

- To demonstrate the significance of selecting a tree with 11 terminal nodes, we can force `rpart()` to generate a full tree by setting `cp = 0` (i.e., no penalty results in a fully grown tree).

```
set.seed(12345)
carseats.dt<- rpart(
    formula = High~.-Sales,
    data    = Carseats[train,],
    method  = "anova",
    control = list(cp = 0, xval = 10) ## xval is number of CV
)

plotcp(carseats.dt)
abline(v = 4, lty = "dashed")
```



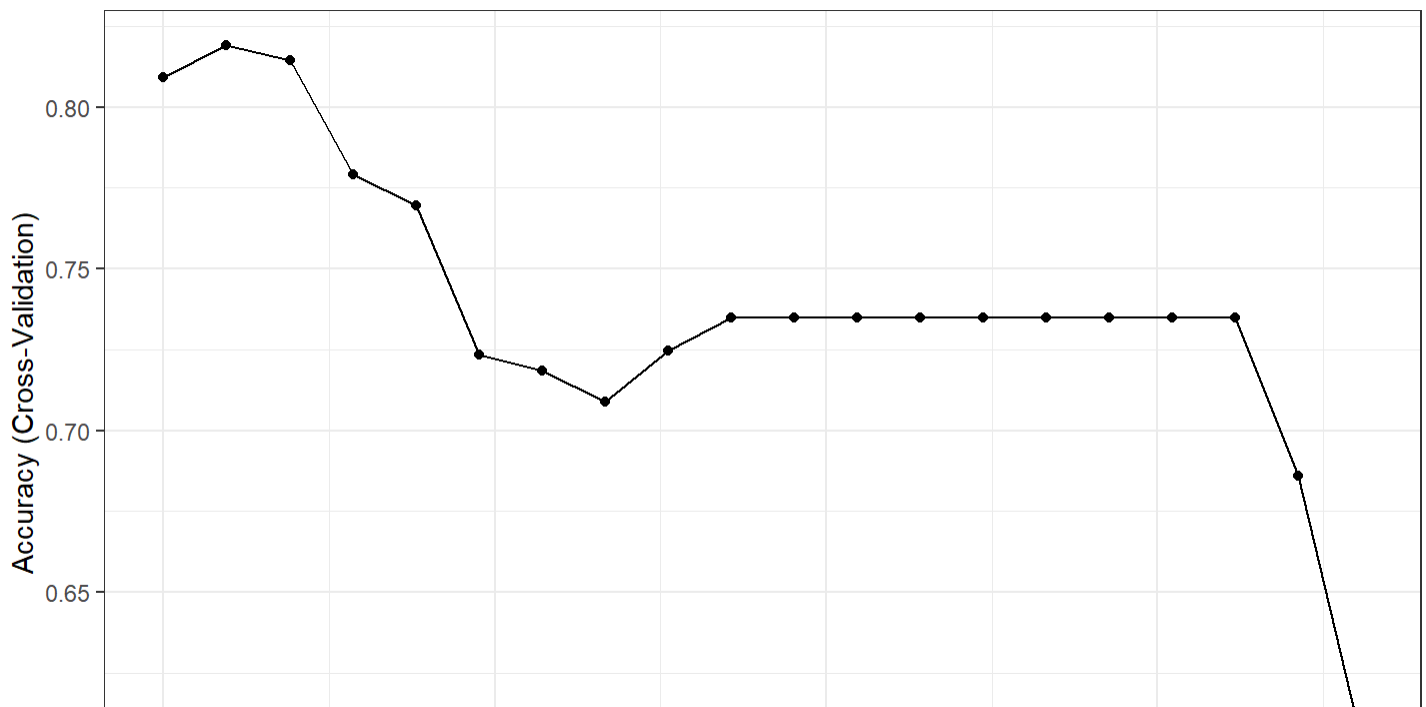
More on `rpart` tuning

- The default behavior of rpart() includes automated tuning, resulting in an optimal subtree with a total of 10 splits, 11 terminal nodes, and a cross-validated sum of squared errors (SSE).
- Although `rpart()` does not calculate the root mean squared error (RMSE) or other metrics, `caret` can be used for this purpose.
- In both cases, smaller penalties (i.e., deeper trees) are producing better cross-validation results.

Using caret

- Cross-validated accuracy rate for the 20 different α parameter values in our grid search.
- Lower α values (deeper trees) help to minimize errors.

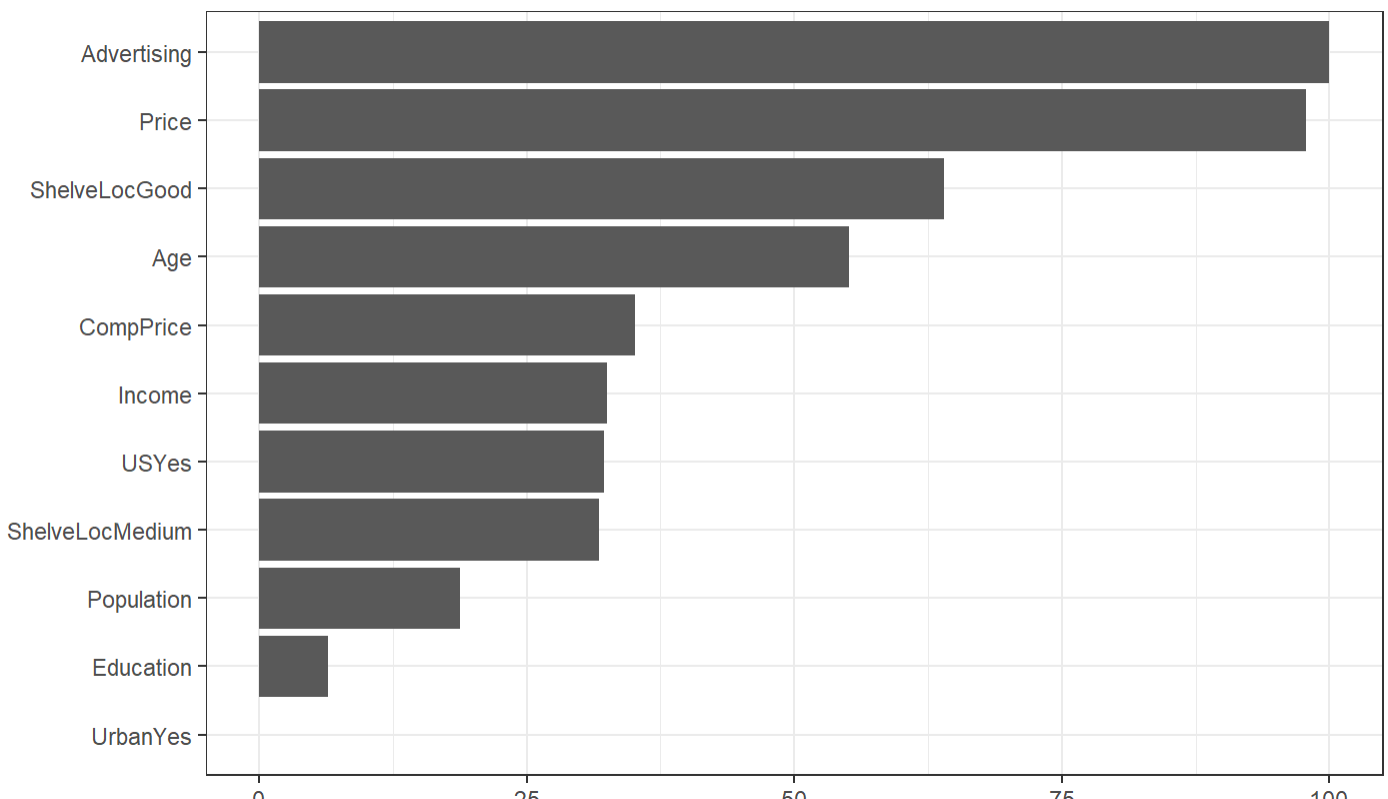
```
carseats.dt2<- train(
  High ~ .-Sales,
  data = Carseats[-train,],
  method = "rpart",
  trControl = trainControl(method = "cv", number = 10),
  tuneLength = 20
)

ggplot(carseats.dt2) + theme_bw()
```



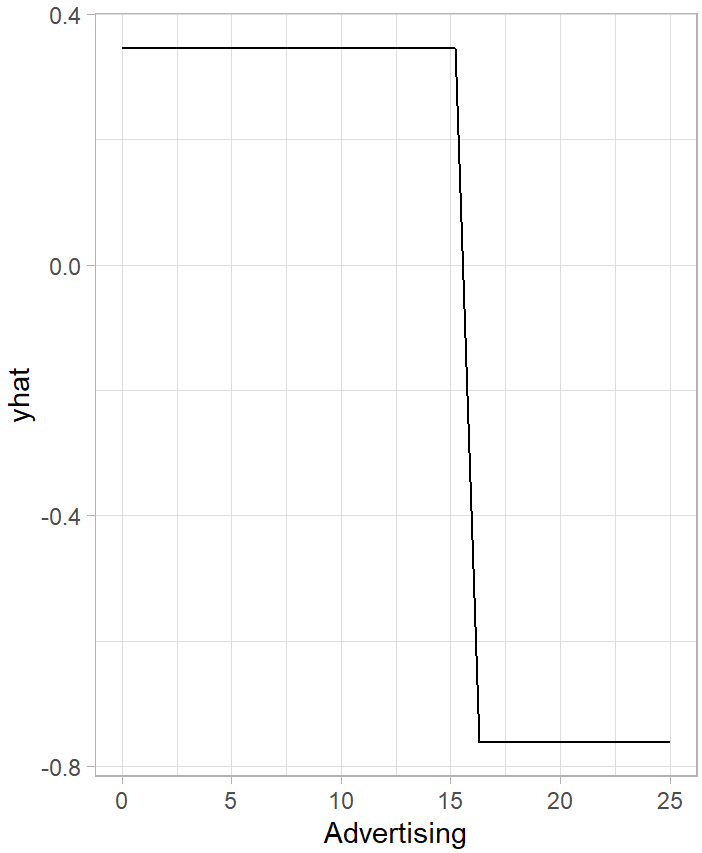
Variable Importance

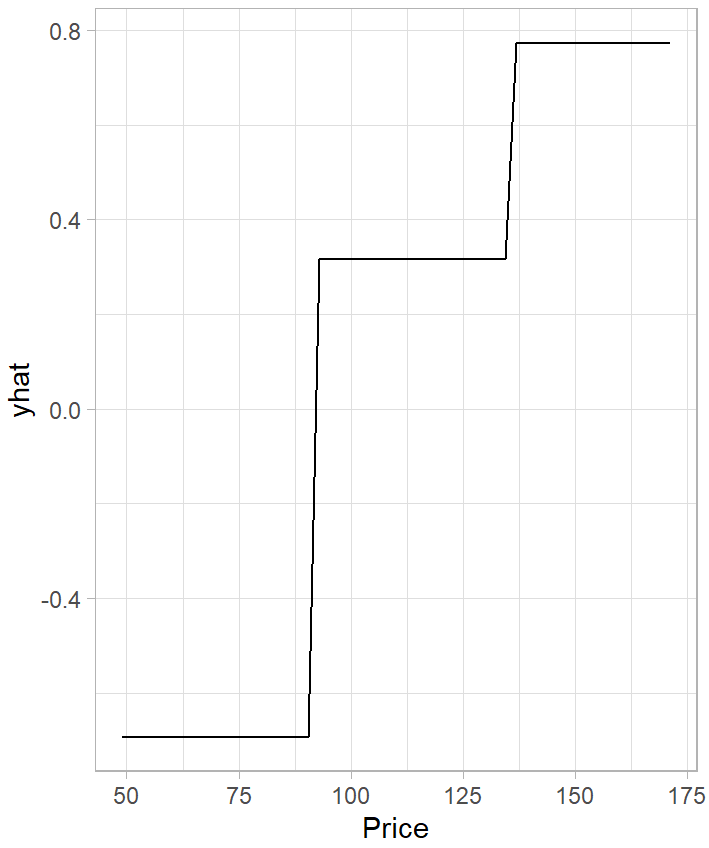
- To determine the importance of features in a decision tree, the reduction in the loss function (e.g., SSE) due to each variable at each split is recorded.
- In some cases, a variable may be used multiple times in a tree, and its total reduction in the loss function across all splits is summed to calculate the total feature importance.
- When using caret, these values are standardized so that the most important feature has a score of 100, and the importance of other features is based on their relative reduction in the loss function.

```
vip(carseats.dt2, num_features = 40, bar = FALSE) + theme_bw()
```



Partial dependence plots





How does this compare to Logistic?

```
glm.fits=glm(High~.-Sales,Carseats,family=binomial,subset=train)
summary(glm.fits)
```

```
##
## Call:
## glm(formula = High ~ . - Sales, family = binomial, data = Carseats,
##     subset = train)
##
## Deviance Residuals:
##      Min        1Q    Median        3Q       Max
## -2.05629  -0.37761  -0.08149   0.26226   2.41387
##
## Coefficients:
##                  Estimate Std. Error z value Pr(>|z|)
## (Intercept)     -3.532167   3.072193  -1.150 0.250259
## CompPrice        0.133686   0.025790   5.184 2.18e-07 ***
## Income           0.031353   0.009661   3.245 0.001173 **
## Advertising      0.229512   0.059475   3.859 0.000114 ***
## Population       0.002068   0.001882   1.099 0.271797
## Price           -0.145926   0.023472  -6.217 5.06e-10 ***
## ShelveLocGood    7.165462   1.260501   5.685 1.31e-08 ***
## ShelveLocMedium  2.938850   0.750210   3.917 8.95e-05 ***
## Age             -0.066072   0.018683  -3.536 0.000405 ***
## Education       -0.023523   0.096187  -0.245 0.806803
## UrbanYes        -0.233306   0.586017  -0.398 0.690541
## USYes           -0.645051   0.745717  -0.865 0.387034
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##     Null deviance: 269.99  on 199  degrees of freedom
```

Prediction

```
glm.probs=predict(glm.fits,Carseats.test,type="response")
contrasts(as.factor(High))
```

```
##     Yes
## No    0
## Yes   1
```

```
glm.pred=rep("No",200)
glm.pred[glm.probs>.5]="Yes"
table(glm.pred,High.test)
```

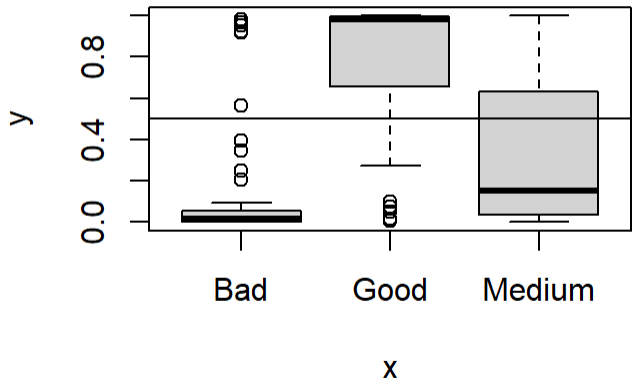
```
##         High.test
## glm.pred  No Yes
##      No  111  11
##      Yes   6  72
```

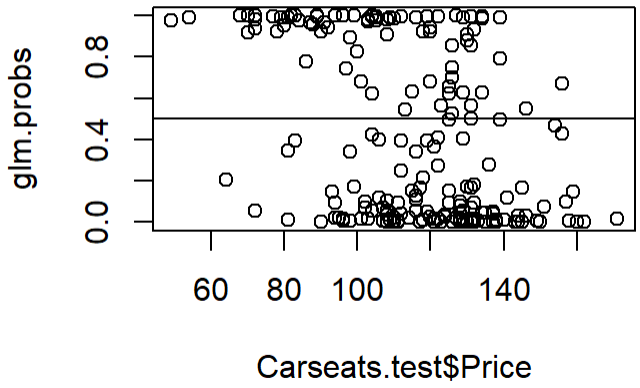
```
mean(glm.pred==High.test)
```

```
## [1] 0.915
```

Significant predictors drive the prediction.

```
par(mfrow=c(1,2))
plot(Carseats.test$ShelveLoc,glm.probs); abline(h=0.5)
plot(Carseats.test$Price,glm.probs);abline(h=0.5)
```





Exercise (not HW)

1. Plot the ROC curve for both logistic and decision tree (with pruning).
2. Also apply k-NN with CV and LDA on the same data-set and compare accuracies.
3. Can you create an artificial data-set where decision tree will do better than logistic or LDA?

Linear Regression vs. Tree

- Linear Regression:

$$f(X) = \beta_0 + \sum_{j=1}^{p} X_j \beta_j$$

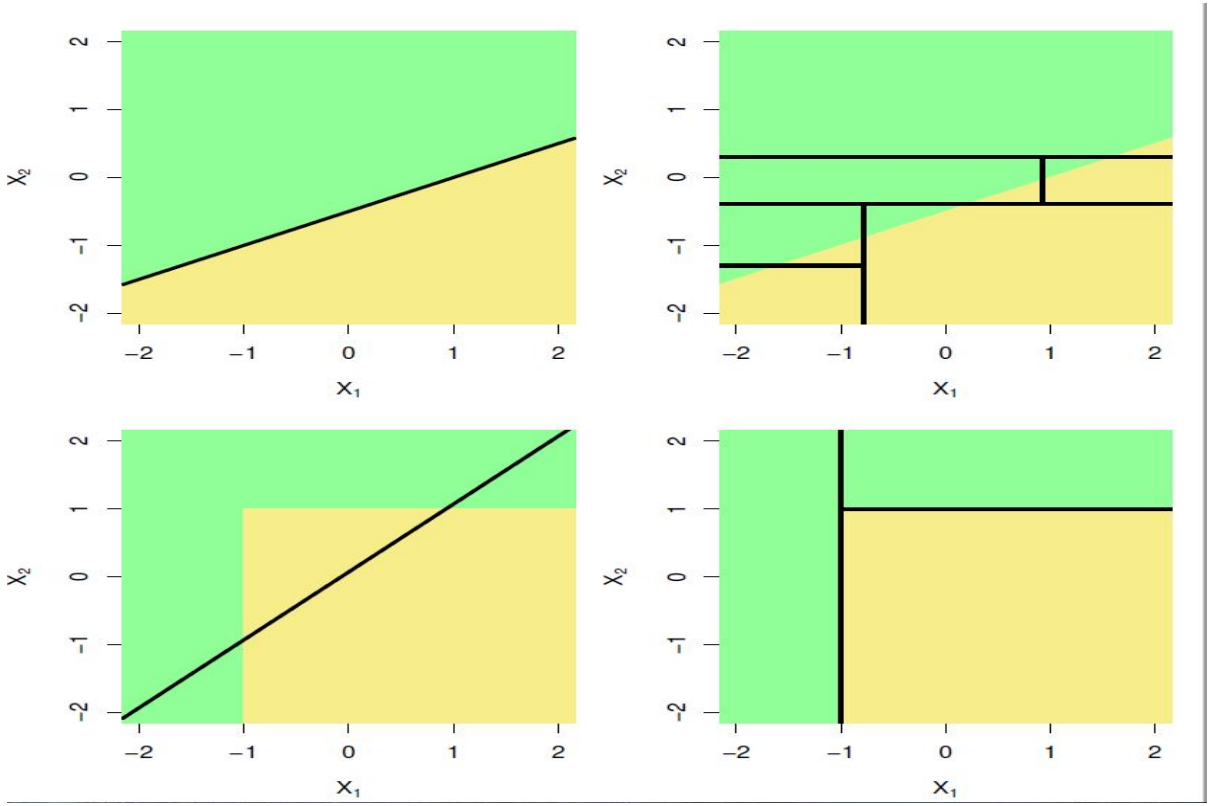
- Regression Tree Method:

$$f(X) = \sum_{m=1}^{M} c_m 1_{(X \in R_m)}$$

  where $R_1, \ldots, R_m$ is a partition of the X-space.

- Which model is better? It depends on the problem at hand.

Which model is better?



If true decision boundary is linear, classical methods work well, if it's non-linear tree-based methods might work better.

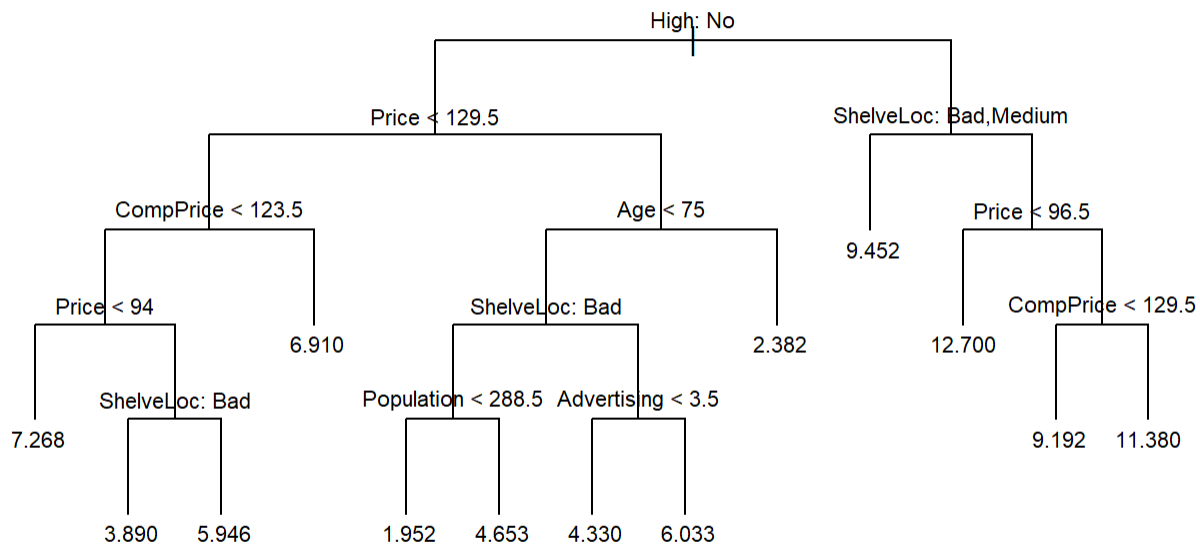
Regression

```
tree.car=tree(Sales~.,Carseats,subset=train)
summary(tree.car)
```

```
##
## Regression tree:
## tree(formula = Sales ~ ., data = Carseats, subset = train)
## Variables actually used in tree construction:
## [1] "High"        "Price"       "CompPrice"   "ShelveLoc"   "Age"
## [6] "Population"  "Advertising"
## Number of terminal nodes:  13
## Residual mean deviance:  1.365 = 255.3 / 187
## Distribution of residuals:
##     Min.  1st Qu.   Median     Mean  3rd Qu.     Max.
## -4.17000 -0.74210 -0.01017  0.00000  0.71470  3.56700
```

Plot Regression Tree

```
plot(tree.car, type = "unif")
text(tree.car,pretty=0,cex=0.7)
```



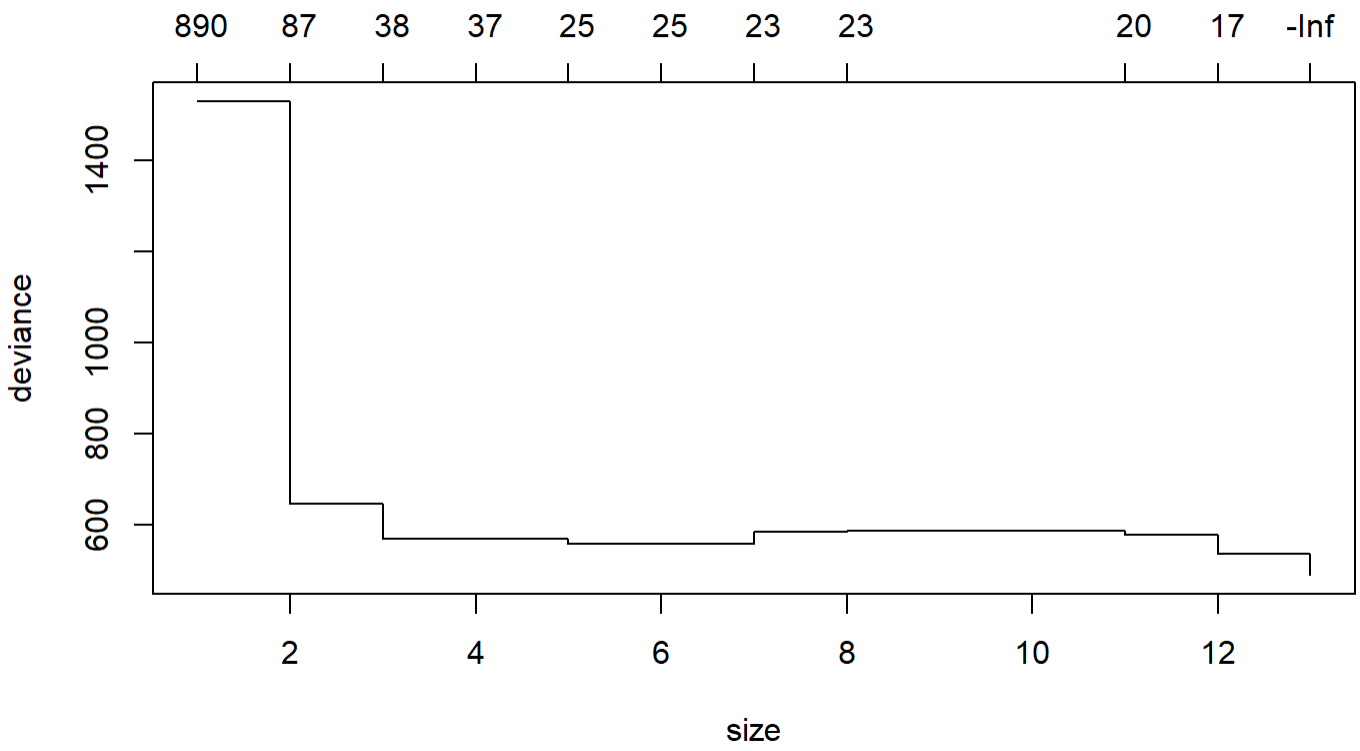
CV for Regression Tree

```
cv.car=cv.tree(tree.car)
cv.car
```

```
## $size
##  [1] 13 12 11  8  7  6  5  4  3  2  1
##
## $dev
##  [1]  491.2872  536.4412  578.0454  587.0062  586.1259  558.0768  558.0768
##  [8]  569.0922  569.7575  646.0310 1530.6952
##
## $k
##  [1]      -Inf  17.39444  19.71259  23.06968  23.45529  24.82006  24.94805
##  [8]  37.04530  37.65622  87.17349 893.23810
##
## $method
## [1] "deviance"
##
## attr(,"class")
## [1] "prune"         "tree.sequence"
```

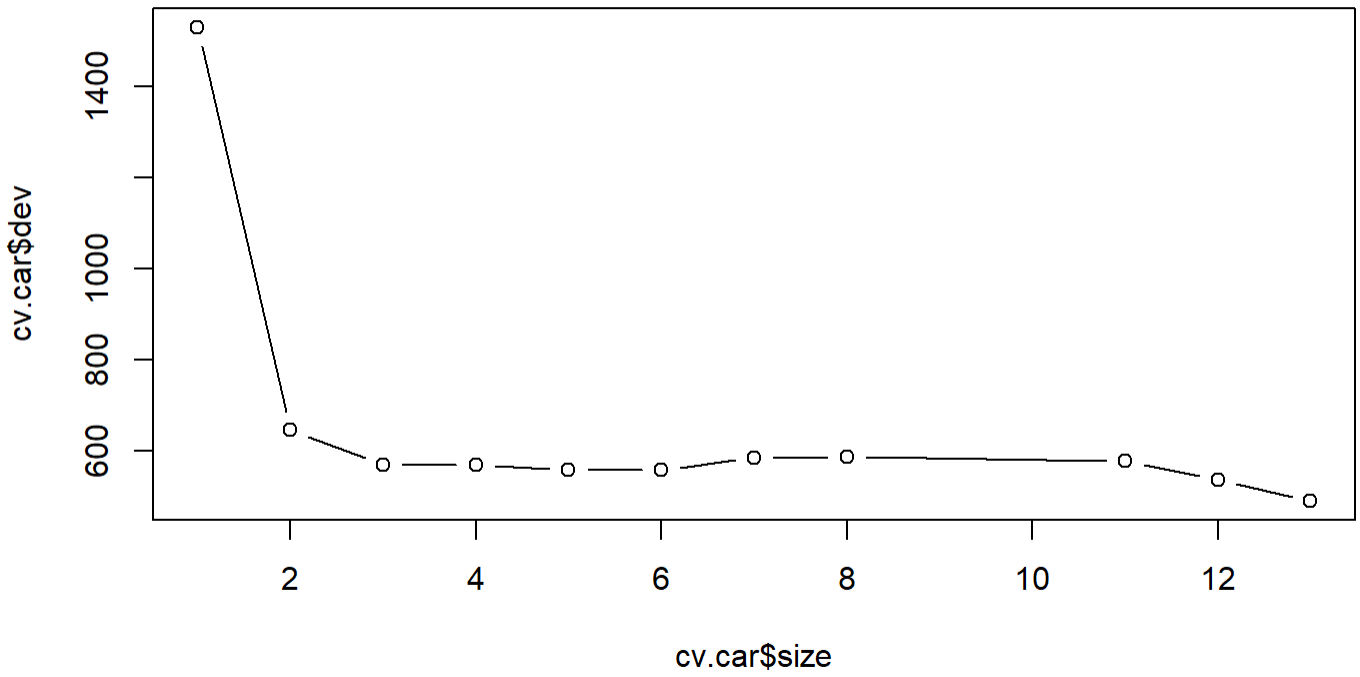
Plot the CV error

```
plot(cv.car)
```



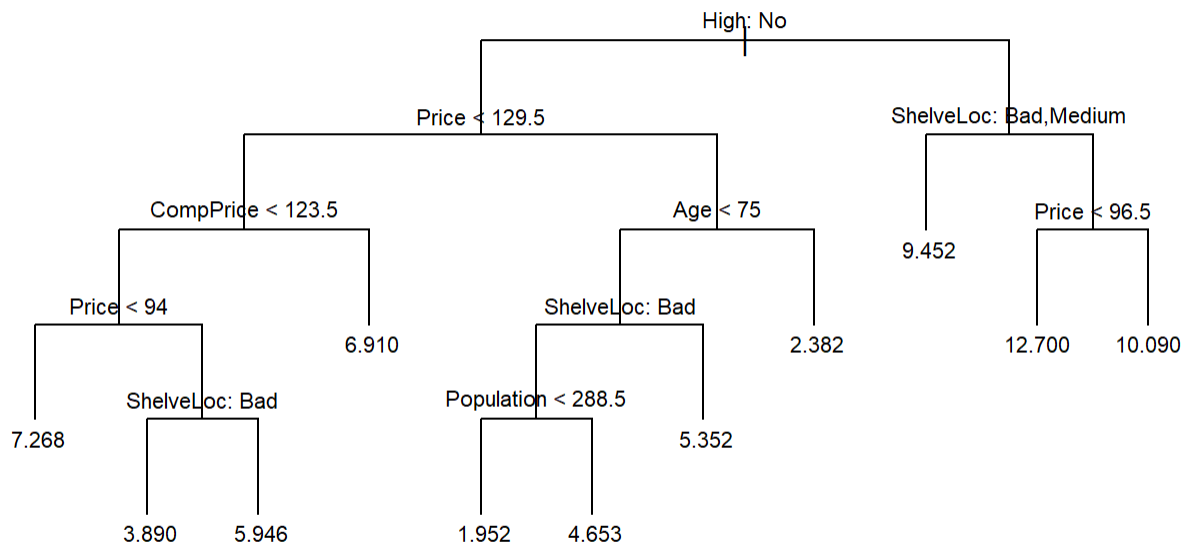
Manually plot the deviance

```
plot(cv.car$size,cv.car$dev,type='b')
```



Prune the Regression Tree

```
prune.car=prune.tree(tree.car,best=9)
plot(prune.car,type = "unif")
text(prune.car,pretty=0,cex=0.7)
```

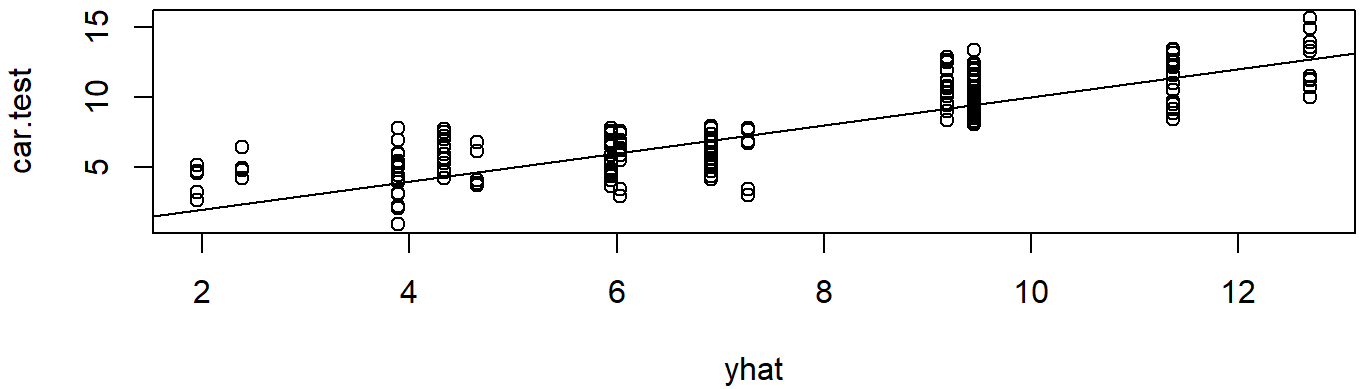


MSE with full tree

```
yhat=predict(tree.car,newdata=Carseats[-train,])
car.test=Carseats[-train,"Sales"]
mean((yhat-car.test)^2)
```

```
## [1] 2.838771
```

```
plot(yhat,car.test); abline(0,1)
```

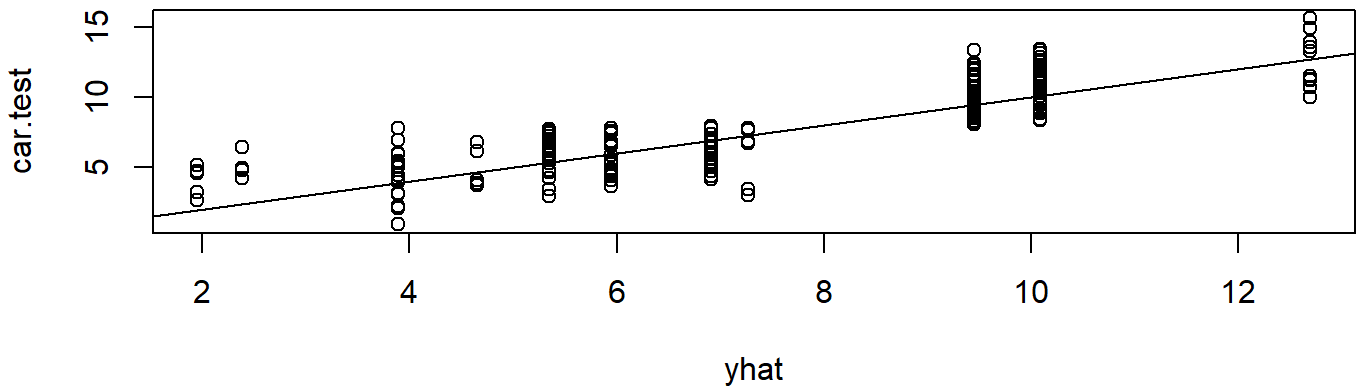


MSE with Pruned Tree

```
yhat=predict(prune.car,newdata=Carseats[-train,])
car.test=Carseats[-train,"Sales"]
mean((yhat-car.test)^2)
```

```
## [1] 2.666092
```

```
plot(yhat,car.test); abline(0,1)
```



Next: Bagging

Bagging and Random Forest

Here we apply bagging and random forests to the `Boston` data, using the `randomForest` package in R.

Random Forest

- Ensemble Learning: Generae many predictors and aggregate their results.
- Two well-known methods: Boosting (Shapire et al., 1998) and Bagging (Breiman, 1996).
- In boosting, successive trees give extra weight to points incorrectly predicted by earlier predictors. In the end, a weighted vote is taken for prediction.
- In bagging, successive trees do not depend on earlier trees - each is independently constructed using a bootstrap sample of the data set.

We use the Boston housing data

```
library(MASS)
str(Boston)
```

```
## 'data.frame':    506 obs. of  14 variables:
##  $ crim   : num  0.00632 0.02731 0.02729 0.03237 0.06905 ...
##  $ zn     : num  18 0 0 0 0 0 12.5 12.5 12.5 12.5 ...
##  $ indus  : num  2.31 7.07 7.07 2.18 2.18 2.18 7.87 7.87 7.87 7.87 ...
##  $ chas   : int  0 0 0 0 0 0 0 0 0 0 ...
##  $ nox    : num  0.538 0.469 0.469 0.458 0.458 0.458 0.524 0.524 0.524 0.524 ...
##  $ rm     : num  6.58 6.42 7.18 7 7.15 ...
##  $ age    : num  65.2 78.9 61.1 45.8 54.2 58.7 66.6 96.1 100 85.9 ...
##  $ dis    : num  4.09 4.97 4.97 6.06 6.06 ...
##  $ rad    : int  1 2 2 3 3 3 5 5 5 5 ...
##  $ tax    : num  296 242 242 222 222 222 311 311 311 311 ...
##  $ ptratio: num  15.3 17.8 17.8 18.7 18.7 18.7 15.2 15.2 15.2 15.2 ...
##  $ black  : num  397 397 393 395 397 ...
##  $ lstat  : num  4.98 9.14 4.03 2.94 5.33 ...
##  $ medv   : num  24 21.6 34.7 33.4 36.2 28.7 22.9 27.1 16.5 18.9 ...
```

Bagging

- Bagging is a special case of a random forest with $m = p$.
- `randomForest()` can perform both random forests and bagging.

```
set.seed(1)
library(randomForest)
train = sample(1:nrow(Boston), nrow(Boston)/2)
boston.test=Boston[-train,"medv"]
bag.boston=randomForest(medv~.,data=Boston,subset=train,mtry=13,
                        importance=TRUE)
```

- The argument `mtry=13` indicates that all 13 predictors should be considered for each split of the tree.
- In other words, bagging should be done.
- `importance=TRUE`: should importance of the predictors be assessed?

Bagging output

```
bag.boston
```

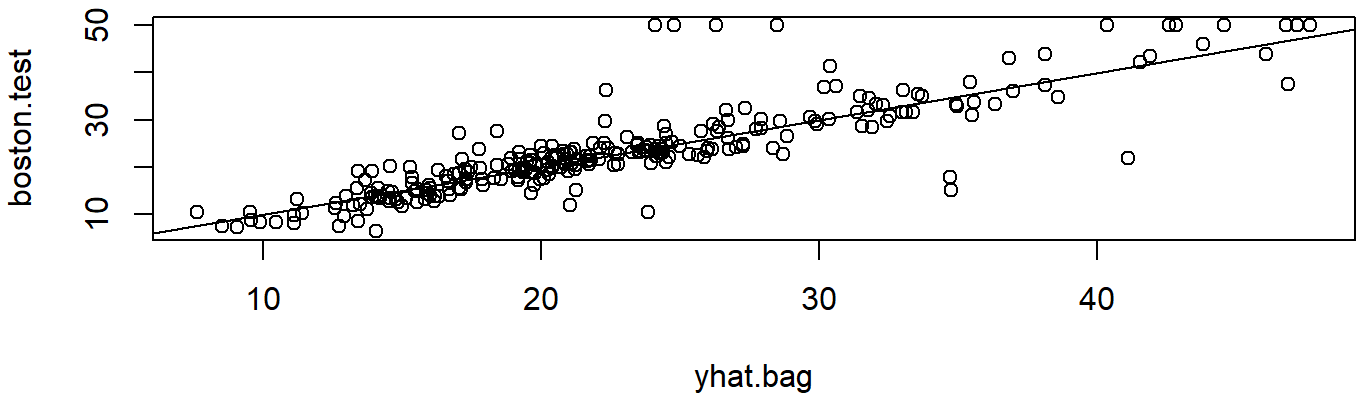
```
##
## Call:
##  randomForest(formula = medv ~ ., data = Boston, mtry = 13, importance = TRUE,      subset = train)
##                Type of random forest: regression
##                      Number of trees: 500
## No. of variables tried at each split: 13
##
##           Mean of squared residuals: 11.33119
##                     % Var explained: 85.26
```

Test error

```
yhat.bag = predict(bag.boston,newdata=Boston[-train,])
mean((yhat.bag-boston.test)^2)
```

```
## [1] 23.4579
```

```
plot(yhat.bag, boston.test); abline(0,1)
```

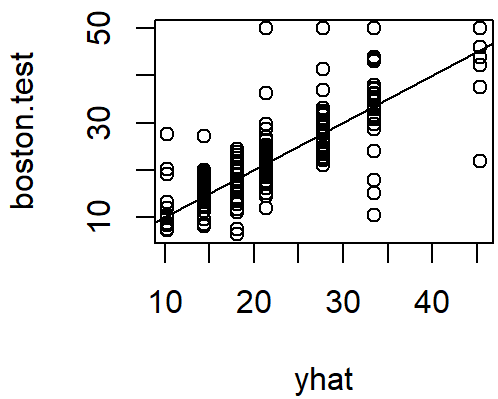


Bagging is much better than using a single Tree

```
tree.boston=tree(medv~.,Boston,subset=train)
yhat=predict(tree.boston,newdata=Boston[-train,])
boston.test=Boston[-train,"medv"]
mean((yhat-boston.test)^2)
```

```
## [1] 35.28688
```

```
plot(yhat,boston.test);abline(0,1)
```



Number of Trees

- By default `ntree = 500`.
- We could change the number of trees grown by `randomForest()` using the `ntree` argument:

```
bag.boston=randomForest(medv~.,data=Boston,subset=train,mtry=13,
                        ntree=25)
yhat.bag = predict(bag.boston,newdata=Boston[-train,])
mean((yhat.bag-boston.test)^2)
```

```
## [1] 22.99145
```

- This is still bagging as `mtry = 13`. What happens to the test error?

Random Forest

- Random forest improves over bagging by a small tweak that decorrelates the trees.
- Just like bagging, we build many decision trees on bootstrapped samples … but when building a tree, **each time a split is considered in the tree**, a random sample of m predictors is chosen from the full set of p predictors.
- A fresh sample of m predictors are chosen for each split and typically $m = \sqrt{p}$ for classification and $p/3$ for regression.

Random Forest

- Growing a random forest proceeds in exactly the same way, except that we use a smaller value of the `mtry` argument.
- By default, `randomForest()` uses $p/3$ variables when building a random forest of regression trees, and $\sqrt{p}$ variables when building a random forest of classification trees.
- Here we use `mtry = 6`.

Random Forest

```
set.seed(123)
rf.boston=randomForest(medv~.,data=Boston,subset=train,mtry=6,importance=TRUE)
yhat.rf = predict(rf.boston,newdata=Boston[-train,])
mean((yhat.rf-boston.test)^2)
```

```
## [1] 19.11609
```

The test set MSE is 19.1160948.

Importance

- Using the `importance()` function, we can view the importance of each variable.

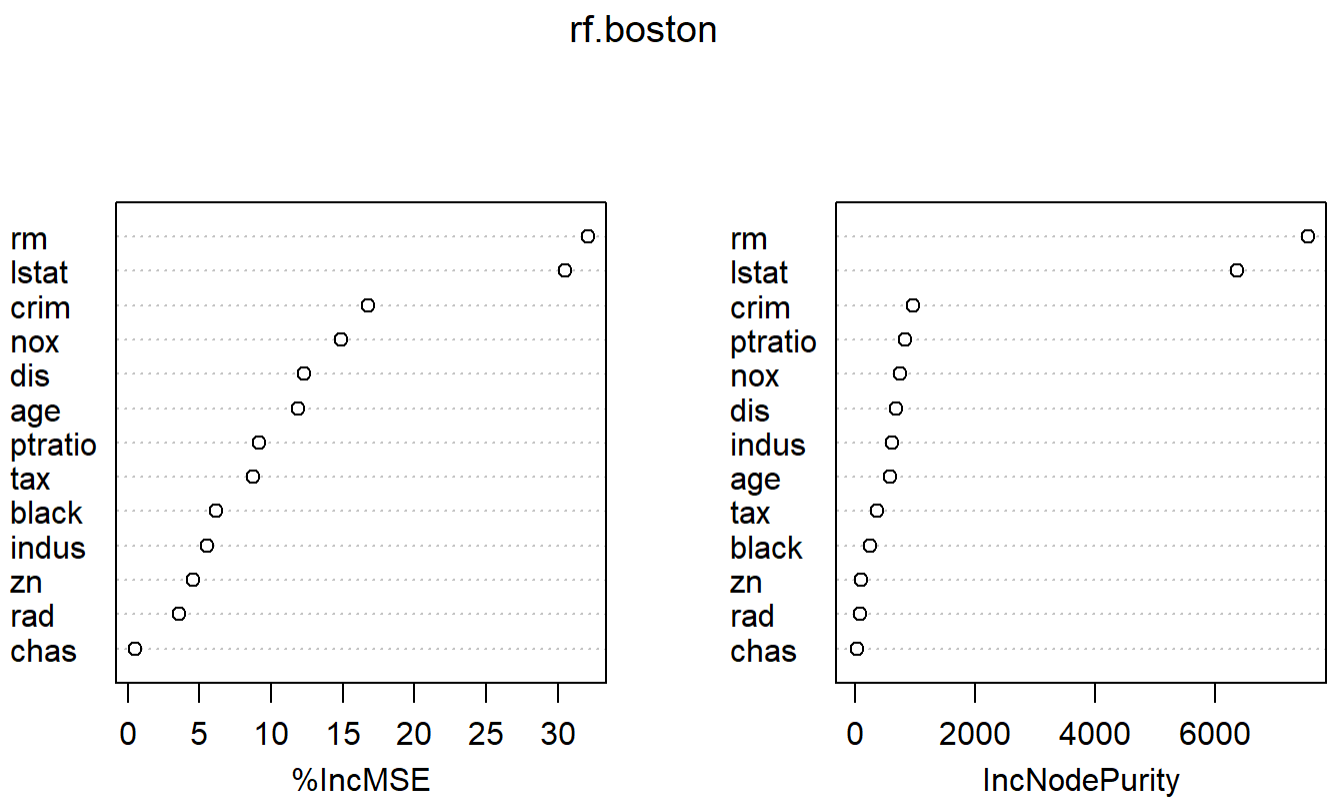
```
importance(rf.boston)
```

```
##            %IncMSE IncNodePurity
## crim    16.7422915     962.64560
## zn       4.5870114     103.88463
## indus    5.5192594     625.85603
## chas     0.4874899      36.22561
## nox     14.8880957     750.14221
## rm      32.1145236    7538.05524
## age     11.8694265     590.10533
## dis     12.2907793     692.36696
## rad      3.5801323      88.23554
## tax      8.7131066     374.44966
## ptratio  9.1472863     836.16087
## black    6.1578417     257.56556
## lstat   30.5232049    6360.09452
```

1. Mean decrease of accuracy in predictions on the out of bag samples when a given variable is excluded from the model
2. Measure of the total decrease in node impurity that results from splits over that variable, averaged over all trees.

Importance Plot

```
varImpPlot(rf.boston)
```

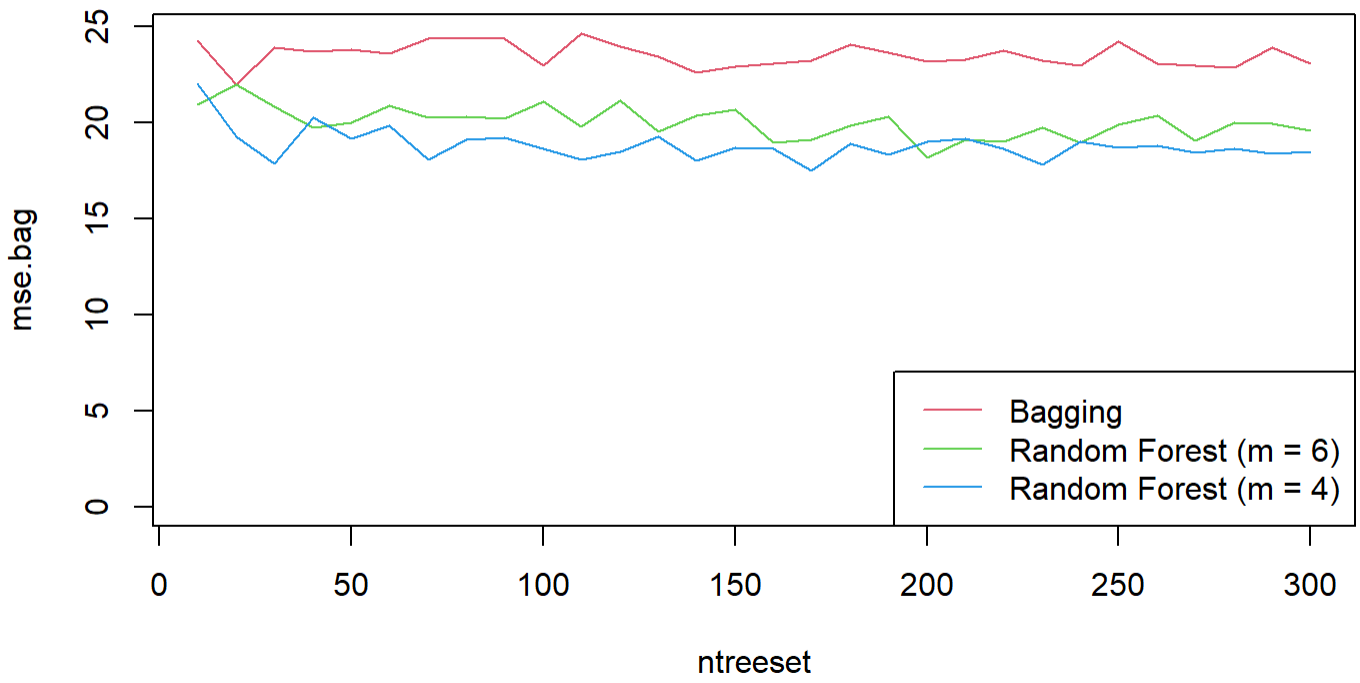


Try 3 different sizes (m = 13, 6 and 4)

```
ntreeset = seq(10,300,by = 10)
mse.bag = rep(0,length(ntreeset));mse.rf1 = rep(0,length(ntreeset));
mse.rf2 = rep(0,length(ntreeset));

for(i in 1:length(ntreeset)){
  nt = ntreeset[i]
  bag.boston=randomForest(medv~.,data=Boston,subset=train,mtry=13,ntree=nt)
  yhat.bag = predict(bag.boston,newdata=Boston[-train,])
  mse.bag[i] = mean((yhat.bag-boston.test)^2)

  rf.boston=randomForest(medv~.,data=Boston,mtry = 6, subset=train,ntree=nt)
  yhat.bag = predict(rf.boston,newdata=Boston[-train,])
  mse.rf1[i] = mean((yhat.bag-boston.test)^2)

  rf.boston=randomForest(medv~.,data=Boston,mtry = 4, subset=train,ntree=nt)
  yhat.bag = predict(rf.boston,newdata=Boston[-train,])
  mse.rf2[i] = mean((yhat.bag-boston.test)^2)
}
```

Plot the test error curve

```
plot(ntreeset,mse.bag,type="l",col=2, ylim = c(0, max(mse.bag)))
lines(ntreeset,mse.rf1,col=3); lines(ntreeset,mse.rf2,col=4)
legend("bottomright",c("Bagging","Random Forest (m = 6)",
                       "Random Forest (m = 4)"),col=c(2,3,4), lty=c(1,1,1))
```



Exercise (at home)

- Using a small value of m in building a random forest will typically be helpful when we have a large number of correlated predictors.
- Apply bagging and random forests to a data set of your choice. Be sure to fit the models on a training set and to evaluate their performance on a test set.
- How accurate are the results compared to simple methods like linear or logistic regression? If it;s a wide data, how does it compare to Lasso or Bayesian methods?
- Which of these approaches yields the best performance?

Try at home

- We'll construct a highly non-linear model.
- Sample p predictors $X_1, X_2, \ldots, X_p$ from a known distribution (normal or uniform), consider

$$Y_i = f(\prod_{j=1}^{p} X_{i,j}) + \epsilon_i, \; i = 1, \ldots, n.$$

- Non-linear model : All X_i's are important here !
- (but you don't know that, you just have y vector and the $n \times p$ matrix X.)
- How does Lasso compare to random forest?

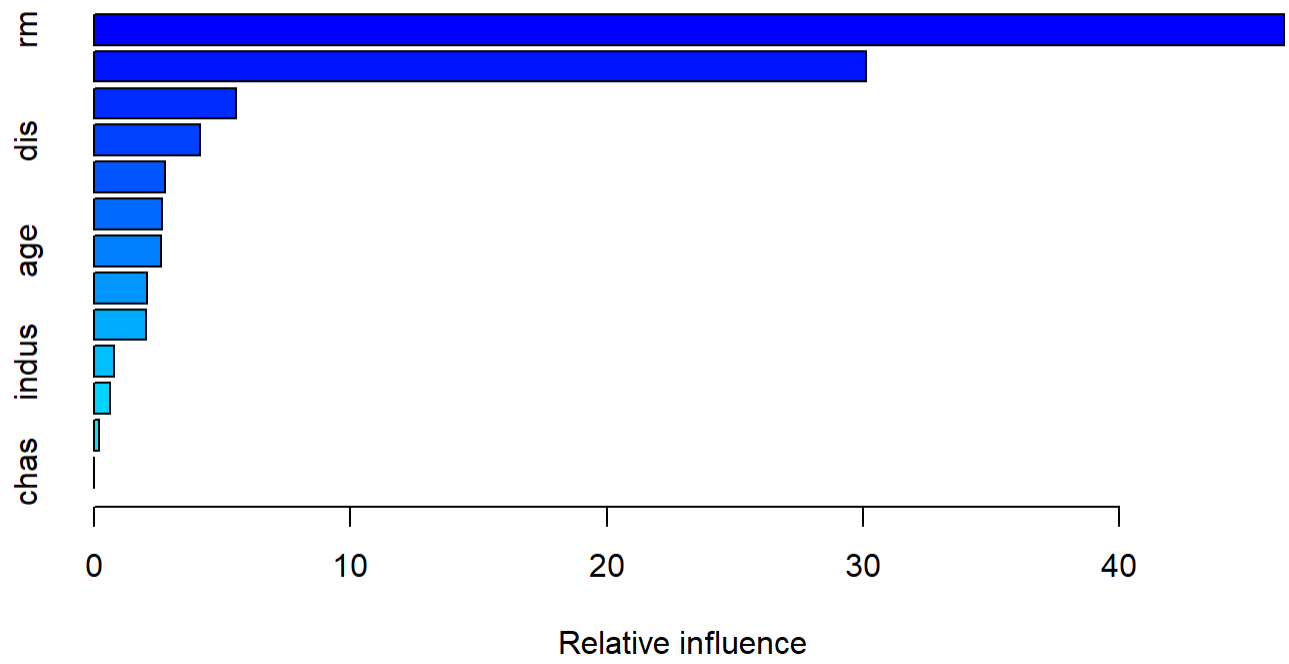
Boosting

- We run `gbm()` with the option `distribution="gaussian"` since this is a regression problem; if it were a binary classification problem, we would use `distribution="bernoulli"`.
- The argument `n.trees=5000` indicates that we want 5000 trees, and the option `interaction.depth=4` limits the depth of each tree.

```
library(gbm)
set.seed(12345)
boost.boston=gbm(medv ~.,data=Boston [train ,],
                 distribution = "gaussian",
                 n.trees =5000,
                 interaction.depth = 4)
```

Summary

```
summary(boost.boston)
```

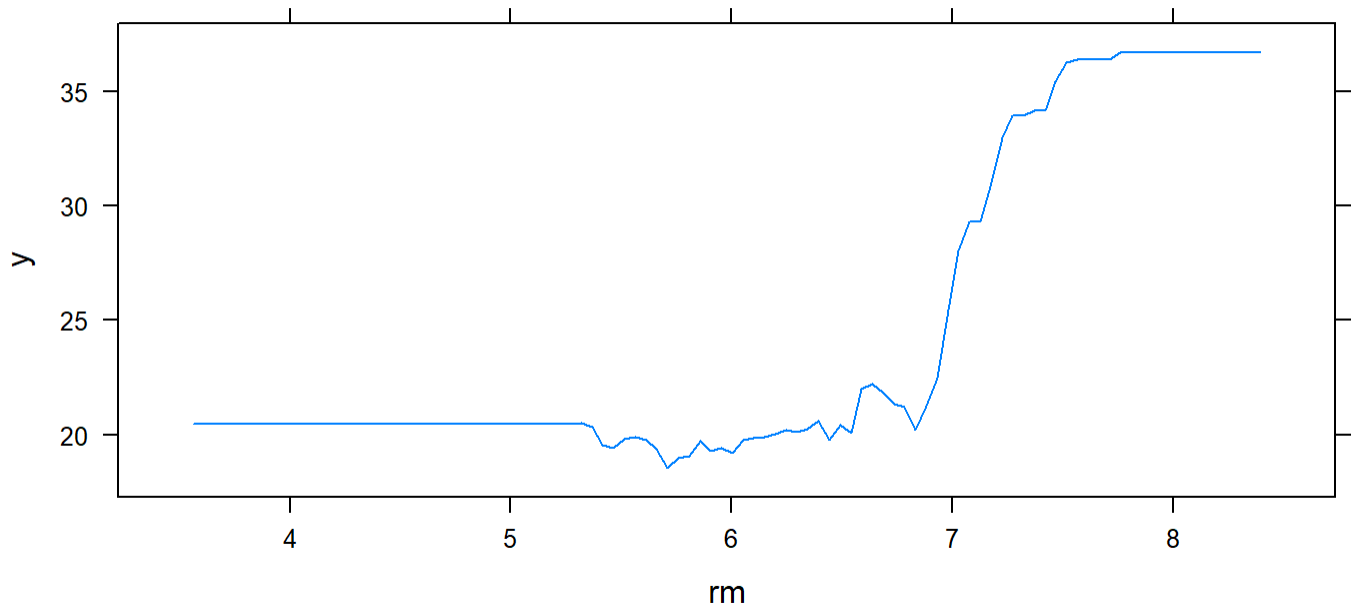



```
##             var      rel.inf
## rm           rm 46.410886279
## lstat     lstat 30.102518273
## crim       crim  5.551760643
## dis         dis  4.136537460
## nox         nox  2.770120310
## black     black  2.648786173
```

Plot

partial dependence plot: illustrate the marginal effect of the selected variables on the response after integrating out the other variables

```
plot(boost.boston, i="rm")
```



MSE

```
yhat.boost=predict(boost.boston,newdata=Boston[-train,], n.trees =5000)
mean((yhat.boost -boston.test)^2)
```

```
## [1] 19.44239
```

Different λ

```
boost.boston=gbm(medv ~.,data=Boston [train ,], distribution=
"gaussian",n.trees =5000, interaction.depth =4, shrinkage =0.2, verbose =F)
yhat.boost=predict(boost.boston,newdata=Boston[-train,],
n.trees =5000)
mean((yhat.boost -boston.test)^2)
```

```
## [1] 19.40553
```